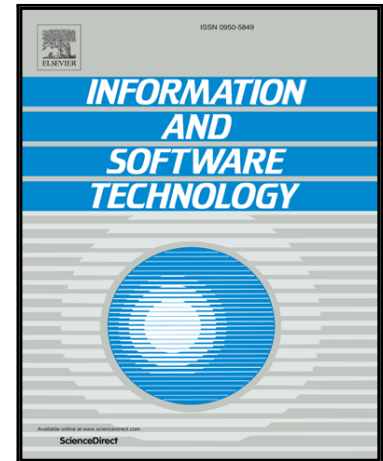


Accepted Manuscript

Improving the Delivery Cycle: A Multiple-Case Study of the Toolchains in Finnish Software Intensive Enterprises

Simo Mäkinen, Marko Leppänen, Terhi Kilamo, Anna-Liisa Mattila, Eero Laukkanen, Max Pagels, Tomi Männistö

PII: S0950-5849(16)30143-4
DOI: [10.1016/j.infsof.2016.09.001](https://doi.org/10.1016/j.infsof.2016.09.001)
Reference: INF SOF 5759



To appear in: *Information and Software Technology*

Received date: 16 December 2015
Revised date: 12 August 2016
Accepted date: 7 September 2016

Please cite this article as: Simo Mäkinen, Marko Leppänen, Terhi Kilamo, Anna-Liisa Mattila, Eero Laukkanen, Max Pagels, Tomi Männistö, Improving the Delivery Cycle: A Multiple-Case Study of the Toolchains in Finnish Software Intensive Enterprises, *Information and Software Technology* (2016), doi: [10.1016/j.infsof.2016.09.001](https://doi.org/10.1016/j.infsof.2016.09.001)

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Improving the Delivery Cycle: A Multiple-Case Study of the Toolchains in Finnish Software Intensive Enterprises

Simo Mäkinen^{a,*}, Marko Leppänen^b, Terhi Kilamo^b, Anna-Liisa Mattila^b,
Eero Laukkanen^c, Max Pagels^a, Tomi Männistö^a

^a*Department of Computer Science, University of Helsinki, P.O. 68 (Gustaf Hållströmin
katu 2b), FI-00014 University of Helsinki, Finland*

^b*Department of Pervasive Computing, Tampere University of Technology, Korkeakoulunkatu
1, FI-33720 Tampere*

^c*Department of Computer Science and Engineering, Aalto University School of Science,
P.O. Box 19210, FI-00076 Aalto, Finland*

Abstract

Context: Software companies seek to gain benefit from agile development approaches in order to meet evolving market needs without losing their innovative edge. Agile practices emphasize frequent releases with the help of an automated toolchain from code to delivery.

Objective: We investigate, which tools are used in software delivery, what are the reasons omitting certain parts of the toolchain and what implications toolchains have on how rapidly software gets delivered to customers.

Method: We present a multiple-case study of the toolchains currently in use in Finnish software-intensive organizations interested in improving their delivery frequency. We conducted qualitative semi-structured interviews in 18 case organizations from various software domains. The interviewees were key representatives of their organization, considering delivery activities.

Results: Commodity tools, such as version control and continuous integration, were used in almost every organization. Modestly used tools, such as UI testing and performance testing, were more distinctly missing from some organizations. Uncommon tools, such as artifact repository and acceptance testing, were used only in a minority of the organizations. Tool usage is affected by the state of current workflows, manual work and relevancy of tools. Organizations whose toolchains were more automated and contained fewer manual steps were able to deploy software more rapidly.

Conclusions: There is variety in the need for tool support in different development steps as there are domain-specific differences in the goals of the case organizations. Still, a well-founded toolchain supports speedy delivery of new software.

*Corresponding author

Email addresses: simo.makinen@cs.helsinki.fi (Simo Mäkinen),
marko.leppanen@tut.fi (Marko Leppänen), terhi.kilamo@tut.fi (Terhi Kilamo),
anna-liisa.mattila@tut.fi (Anna-Liisa Mattila), eero.laukkanen@aalto.fi (Eero
Laukkanen), max.pagels@alumni.helsinki.fi (Max Pagels),
tomi.mannisto@cs.helsinki.fi (Tomi Männistö)

Keywords: continuous deployment, continuous delivery, software development tools, deployment pipeline, agile software development

1. Introduction

Today, software development has evolved into a high-paced business tackling challenges of demanding contexts, changing customer requirements and the increased need for short development cycles and fast time-to-market. At the same time, software development organizations should be able to produce high-quality software while maintaining an innovative edge.

In a quest to answer these challenges, software development organizations have turned to agile practices such as Kanban [1], Scrum [2] and Extreme Programming (XP) [3] due to their promise of speed of delivery, ability to adapt to changing requirements and improved software quality. The leading principle of agile is to “satisfy the customer through early and continuous delivery of valuable software” [4]. Some companies, such as Amazon, Facebook and Google [5] being some of the most famous examples, have adopted agile practices to the extent where software is released to the customers continuously in a rapid pace. Still, many are yet to adopt this kind of agility to its fullest.

Continuous deployment refers to the practice of deploying new software and features to the customers as it gets developed instead of making less frequent timed releases [6]. The one mandatory aspect of continuous deployment is a pipeline without any interruptions all the way from code to delivery. Hence, in order to achieve continuous deployment, a software company must set up an automated toolchain and establish tool-related workflows [7]. A properly documented and easy to set up toolchain enables a company to get development going rapidly and makes speedy deployment of new software to the customers possible.

In this paper, we investigate the current state of the practice in Finnish software organizations regarding their workflows and toolchains. We focus on the current toolchains organizations have in place and how automated the different stages of development are. In addition, we investigate why tooling is not used for some phases and does the missing tooling have effect on the speed of delivery. We conducted interviews with the representatives of 18 Finnish software organizations. The organizations were of different sizes and represented several business domains to get a comprehensive view on the phenomenon.

The rest of the paper is structured as follows: Section 2 discusses the background and related work. Section 3 introduces the research methodology. Section 4 describes the modern software development toolchain and its elements, constructed in this study. Section 5 gives the results from the conducted interviews. Section 6 draws comparisons with the maturity of the toolchain and the deployment times. The results introduced in Sections 5 and 6 are then discussed in Section 7 where the validity threats for the study are also explored together with future directions. Section 8 summarizes the findings and concludes the paper.

2. Background

During the past two decades software development has transformed from a rigid and documentation-heavy waterfall approach to lean and agile development models. Lately, the cross-functional teams and the ability to rapidly deliver new software to end-users have been noted as especially important practices. The concept of DevOps, where developers and operations work closely together, is fused together with continuous deployment into the day-to-day software work. This section presents the lean principles from which the modern-day software development practices have been partly derived from, describes the terminology related to the continuous forms of software engineering, and positions the article against the backdrop of related work.

2.1. *Lean and DevOps*

Modern software development relies on a versatile set of agile practices which in turn have been heavily influenced by lean development principles. Lean itself originates from manufacturing [8] while the concept of lean software development was introduced by Poppendieck and Poppendieck [9]. The Poppendiecks' base lean software development on seven key principles: 1) Eliminate waste, 2) Amplify learning, 3) Decide as late as possible, 4) Deliver as fast as possible, 5) Empower the team, 6) Build integrity in, and 7) See the whole. Continuous software engineering [10] is an emerging trend taking a holistic approach on continuous development – incorporating these seven lean principles to the development.

One development approach that relies on teams' ability to deploy features continuously in an automated manner, and is gaining increasing popularity, is DevOps [11, 12] – a portmanteau of developers (Dev) and operations (Ops). In DevOps, developers and information technology professionals (operations) work together as a team instead of being separated in so-called 'information silos'. DevOps relies on practices that help to shorten the time between committing a change and its deployment to normal production use while maintaining high software quality. Thus, it shares many of the goals as continuous software engineering in general – improved communication, more intensive collaboration, and faster delivery of high quality software – as well as builds on top of a solid toolchain from code to production.

2.2. *Continuous Integration, Delivery and Deployment*

At first, a few words on terminology. There is a multitude of terms and concepts involved in continuous software engineering [10]. For the purpose of this paper, the distinction between continuous integration, continuous delivery and continuous deployment is crucial. In continuous integration [13], developers send their changes frequently to version control repositories which trigger tests on continuous integration servers whereas in continuous delivery [14] the steps include additional, automated, actions to prepare a release-ready software version. Typically, the actual release to the production system is not fully automated and requires manual intervention with continuous delivery but in continuous deployment [6] the last mile is also automated.

One of the principles of lean development is “deliver as fast as possible” [15]. In this spirit, the typical agile development process incorporates the concept of continuous integration (CI) [13]. To enable continuous integration, the

developers in the team integrate their work frequently under version control. Continuous integration also includes the practice of automated tests in order to get frequent, fast feedback to the development team. Even though there are different levels of automation and flavors to what gets included into continuous integration, the concept is still relatively well understood and defined.

The difference between continuous delivery and continuous deployment is less well defined. In his blog, Fowler [14] defines continuous delivery “as a software development discipline where you build software in such a way that the software can be released to production at any time.” Neely and Stolt [16] define continuous delivery on the same lines as “the ability to release software whenever we want”. Thus, in continuous delivery, a new software version is not necessarily deployed automatically to the users but it is kept deployable all the time during development of new features. Continuous deployment requires continuous delivery, but takes it a step further and deploys every commit that passes the tests must be delivered automatically to production [6]. For the scope of this paper we will use the definition: continuous deployment is the practice of always deploying new software to production as it gets built.

Humble and Farley [17] describe that continuous delivery is achieved with a stage-gate process called deployment pipeline, consisting of *commit stage*, *acceptance test stage* and *other testing stages*. This pipeline starts similarly to traditional continuous integration. Developers make changes to source code and each change committed to the version control system will trigger the commit stage of the pipeline. In this stage, software is typically compiled, unit tested and a build artifact is produced. Often, static code analysis and code metric collection is also done. The commit stage should be fast and able to catch the most severe errors. Next, the acceptance test stage is executed. It is still automatic, but executed in a more production-like environment and can take a longer time to complete. The purpose of this stage is to validate that the software fulfills its acceptance criteria and could be released to the users. After passing the acceptance test stage, the software can go through other testing stages which can be manual, such as user acceptance testing or capacity testing. Manual testing stages should, however, be cut down to the bare minimum because they stop or slow the flow in the deployment pipeline. After passing all the testing stages, the software can be released to production.

Based on the recommendations of Humble and Farley, the deployment pipeline should ideally be automated as much as can be, or at the very least, semi-automated. For example, even if deployment would not happen automatically for each successful integration, it should be possible to execute the deployment with a single push of a button. Thus, the recommendation is that all manual work is minimized, leading to the logical conclusion that proper toolchains are essential to continuous delivery.

Ståhl and Bosch studied what effects does continuous integration have [18] and how their differences can be explained with differences between continuous integration implementations. Later, they developed a meta-model to describe differences in continuous integration implementations [19]. They have discovered that (1) continuous integration practices vary between companies and even between projects and (2) what companies call continuous integration is not as continuous as usually understood [20]. Because of (2), Ståhl and Bosch suggest calling the integration practices “automated integration flows”.

This study is similar to the study of Ståhl and Bosch in the sense that in both

140 studies, toolchains for automated software development are studied. However, the focus of Ståhl and Bosch is on continuous integration practice, whereas in our study, the focus is on the whole, continuous deployment.

Leppänen et al. [21] focused on the potential benefits and challenges of continuous deployment in a multiple-case study of 15 industry companies. They found out from the interviews that although not all companies aimed for fully automated continuous deployment, frequent releases could potentially help the companies to build better products for the customers and be more reactive in development. Social factors within companies or attitudes in the field could prevent the adoption of continuous deployment practices. Technical matters such as the size and complexity of projects, and the need to perform manual testing stages were seen also to be in the way of continuous deployment. The study presented in this article is based partly on the data from the study of Leppänen et al. [21] but the emphasis here is on the overall development process utilized in the companies and the underlying tool infrastructure which were not covered in the original study.

155 Claps et al. [22] studied continuous deployment with a single exploratory case study strategy. Through interviews, they discovered a total of 20 technical and social continuous deployment adoption challenges. To mitigate these, they also proposed strategies which were adopted by the case company. Multiple challenges mentioned by Claps et al., such as hardware and software infrastructure, deploying the product, seamless upgrades, continuous integration process, testing, source code control, shorten customer feedback and team coordination may need tool support to be overcome properly. Thus, their results support the argument about the importance of toolchain.

3. Research Methodology

165 In this section, we formalize the research methodology and characteristics of the multiple-case study, describe the study design, detail the case companies, describe how data was collected through interviews and analyzed with thematic analysis, and finally synthesized into the research results.

3.1. Research Methodology and Study Design

170 Our multiple-case study [23] is an exploratory study of the toolchains in Finnish software-intensive organizations interested in transitioning towards a more rapid deployment rate – up to continuous deployment. Our main aim is to ascertain the current state of the practice with respect to continuous deployment in the Finnish IT field. For this study, our formal research questions are:

- 175 • **RQ1:** Which toolchains are used to support software delivery?
- **RQ2:** What are the reasons behind the gaps in the toolchains?
- **RQ3:** What possible implications do toolchains have on the speed of software delivery?

180 In order to help researchers consider the research process as a whole, we use the framework by Wohlin and Aurum [24] to characterize our research:

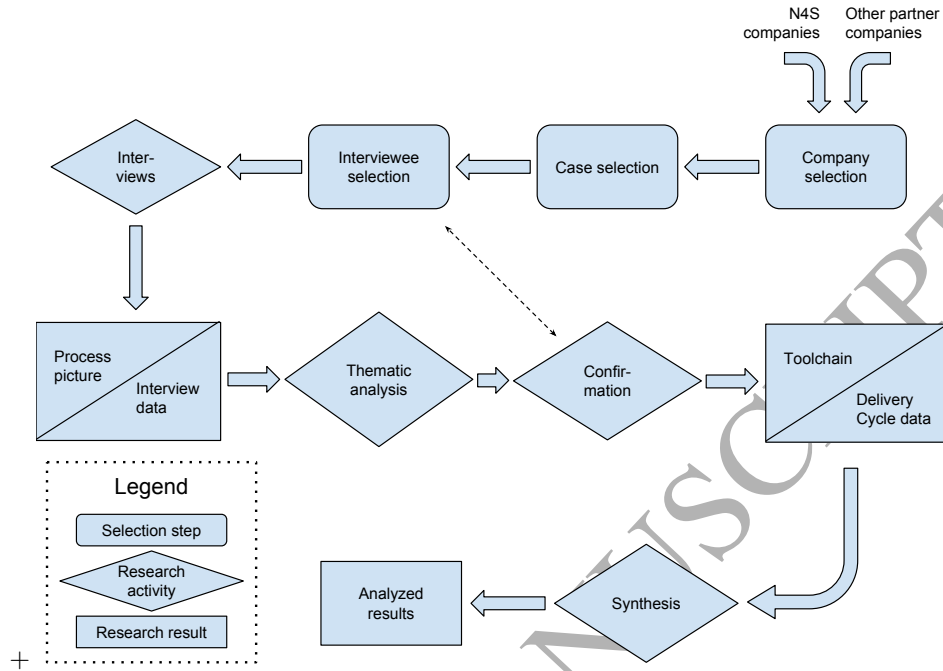


Figure 1: Research process of the study. Different steps of the process are explained in Sections 3.2–3.5.

- Our research outcome is *basic research* instead of applied research, because we investigate a problem instead of proposing a solution.
- We use *inductive* research logic instead of deductive, because we do not have an existing theory to test. Rather, our contribution can be used for theory-building.
- Our research purpose is *exploratory*, as our research topic is quite new and there are not many existing studies.
- Our research approach is *interpretivist*, because especially RQ2 and RQ3 would be difficult to study under the positivist lens.
- Our research process is *mixed approach*, because RQ1 and RQ2 are based on qualitative data, whereas RQ3 uses quantitative approach.
- Our research methodology is *case study*, because it allows deeper investigation of the phenomenon than surveys.
- Our data collection method is *interviews*, because it is the most feasible option to study multiple cases, especially for questions RQ2 and RQ3.
- Our data analysis method is *thematic analysis*, because it is well suited for RQ1 and RQ2. For RQ3, we also used descriptive quantitative measures.

To answer the research questions, we followed the research process depicted in Figure 1. In the following sections, we describe the process more specifically.

Case	Domain	Personnel	Team Size	Platform	Primary Technologies
C1	Consulting	80	8	Web	Java
C2	Consulting	80	7	Web	Java, Javascript
C3	Consulting	250	<10	Web	Clojure, Javascript
C4	Consulting	180	3	Web	Javascript, Scala
C5	Consulting	100	10	Embedded	C++
C6	Embedded Systems	500	30	Embedded	Proprietary
C7	Industrial Automation	15000	50	Embedded	C++
C8	Mobile Games	7	7	PlayStation	Platform specific
				Portable, Apple iOS, Google Android, PC	
C9	Mobile Games	9	4	Apple iOS	Unity, Scala
C10	Mobile Games	9	3	Apple iOS, Google Android, Windows Phone	C++ (cross-compiled)
C11	Telecom	>10000	100–300	Network	Java
C12	Telecom	>10000	50–100	Network	C++
C13	Telecom	>10000	350	Network	C++
C14	UI Framework	80	7	Web	Java
C15	UI Framework	1000	200	Cross-platform	C++
C16	Web Services	7	7	Web	Ruby on Rails, JavaScript
C17	Web Services	8	3	Web	Ruby on Rails
C18	Web Services	90	30–50	Web	Java, Scala

Table 1: Background information of the selected case organizations.

3.2. Company and Case Selection

The case selection of the study was made with a twofold selection criteria. Our larger population of interest comprises Finnish software-intensive companies and enterprises interested in the benefits afforded by faster software delivery. Inside this population our selection comprises companies i) participating in Need for Speed (N4S) [25], a Finnish research program focusing on rapid value delivery, and ii) other companies known to researchers as interested in the subject. The Need for Speed research program is a four year program running from 2014 to 2017 with around 40 program partners. Majority of the program partners, thirty or so, are industrial organizations and the rest are universities or research institutes. Partners in the program engage in joint industrial research and development endeavors in the area of rapid value delivery and this article is a result of such work. By joining the program, the industry partners have shown interest and commitment to improve their capabilities towards more real-time operations.

We approached prospective program partners and other known companies in person and via e-mail, asking of their interest to participate in the study. The companies could freely choose whether to participate in the study. As such, selection at the organization level can be considered self selection or convenience selection. However, within this context, companies were purposefully chosen to ensure variability in terms of domain and organization. Almost the same selection of companies was used in a previous study [21]. Our study includes four additional cases (C6, C12, C17, C18) but excludes one case because it lacked detailed tooling information (Company O in [21]).

Most of the selected companies were small or medium sized enterprises with less than two hundred employees. Several well-established companies in the study were substantially larger, employing over ten thousand workers each. Also, the industry sectors of the companies differed. The consultancies were engaged with contracted customer projects where the technologies varied per project whereas some case companies had a more specific focus. Companies in the mobile game industry and telecommunications formed clearly identifiable

groups. The web was a popular choice as a platform, but some companies also targeted embedded and mobile platforms. These included embedded Linux and consumer-oriented mobile operating systems like Apple iOS or Google Android. Software in these specific cases was typically programmed in Java, C++ or Scala. In the web domain, Javascript and Ruby on Rails were mentioned as was the functional programming language Clojure. Characteristics of the companies can be seen from Table 1. It should be noted that the cases C11 and C12 were from the same company, but from different products. That is why we refer to the cases as organizations instead of companies.

From this group of selected organizations, we made a selection of interviewees. We requested to interview a technically well-versed representative of the development organization that was their most advanced in terms of adoption of rapid delivery practices. Organization representatives were asked to evaluate whether other teams in the organization were using similar software processes and methods, and the rationale for potentially differing practices but the relative experience or expertise of the team was not judged further. Heterogeneous practices are possible especially in large organizations so to make the responses concrete, the unit of analysis was chosen to be the projects or products engineered by the team of which the interviewee had direct experience. The interviewees were software developers, architects, quality assurance personnel and team leaders who worked mostly in small developer teams of less than ten people. The teams in the telecom organizations were quite a bit larger than the more typical teams and, in addition, were geographically distributed across multiple sites.

3.3. Interviews

This study utilizes the interviews conducted in a previous study [21]. Same interview records and notes were analyzed, but four additional interviews were done for this study alone. Also, the analysis in this continuation study focuses on toolchains, whereas the previous article discussed the motivations, benefits and obstacles of continuous deployment practice. Overall, there were a total of around 50 questions grouped into 9 subtopics in the original interview protocol and the same questions were used in the additional interviews. Questions about the toolchains in the organizations were part of the broader original study design so the datasets between the studies were compatible without further need for adjustment. The relevant subset of questions for this study is highlighted in Appendix A, while other topics are included there too for completeness to illustrate the other themes covered in the interviews but which were not pertinent to the continuation study.

In order to answer the research questions, we chose surveys in the form of open-ended, semi-structured interviews. Interviews were carried out in Finnish, with two researchers present and primarily one case representative. Mostly, one researcher presented the questions following the interview outline while the other acted as a scribe and took notes. Three separate research teams from the participating universities were involved in conducting the interviews. In addition to basic closed-ended demographic questions, interviewees were asked a variety of open-ended and closed-ended questions on software delivery, see Appendix A. These questions were grouped into subtopics; the data presented in this article pertains to the topic of toolchains.

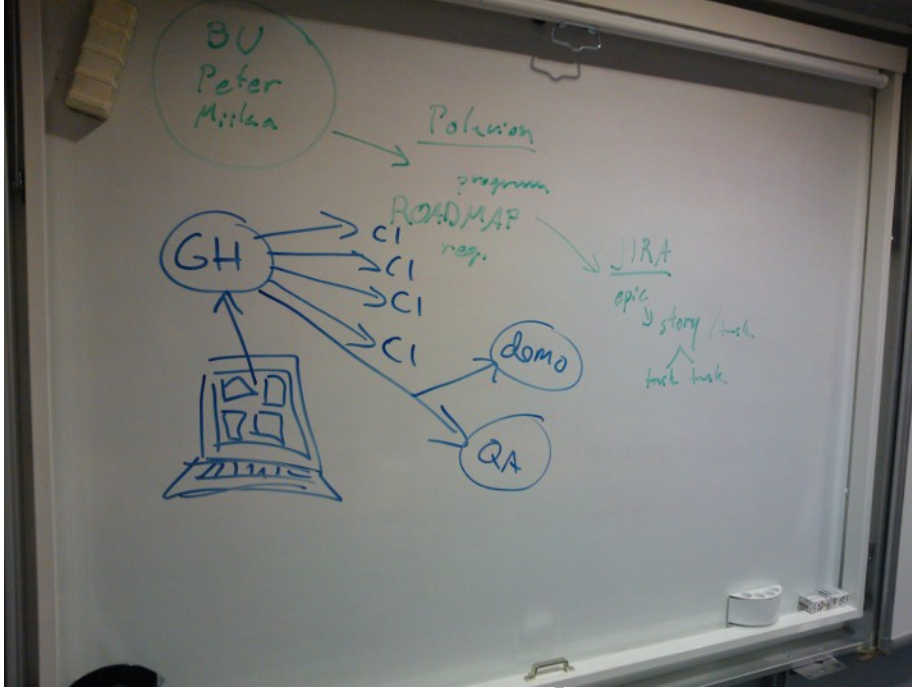


Figure 2: A typical process diagram drawn by an interviewee.

Interviewees were asked to describe the tools they use in daily project activities, and to denote the stage or stages in which tools were used as part of their development workflow. This was conducted by asking the interviewee to draw a picture of their delivery process, see Figure 2 for an example. Qualitative data from the interviews was gathered primarily by taking notes during the interviews. Using audio recordings as support, the notes were revised after each interview in order to ensure completeness. The revised notes were circulated with the researchers not present in the interview.

3.4. Thematic Analysis

The primary analysis method for the data collected in our study is thematic analysis [26]. In thematic analysis, collected data is first examined in order to extract relevant items such as results or other key findings from the data. As the analysis and synthesis progresses, extracted data is coded and assigned to common categories for grouping the coded entities. Similar codes and categories are later associated with more abstract themes which are seen to represent the underlying concepts. Thematic analysis allowed us to approach the data analysis from the ground up by identifying tools and categorizing them into common categories based on their purpose.

Using the revised interview notes, tools used in each organization were coded into themes based on the primary purpose of the tool (i.e. the scope and situations for which it is intended to be used). If the notes were not comprehensive enough, audio recordings were listened again to acquire the needed information. Documentation and public websites of tools were consulted in cases where the

purpose of the tool was unknown to researchers. Thematic analysis of qualitative data is subject to a number validity threats, the specifics of which are described in detail in Section 7.5.

305 Following theme identification, each theme was given a name to reflect a particular software development activity. For each identified tool, there is one or more associated primary activities, as a single tool could belong to multiple activities. The constructed activities are described in the next Section 4. After assigning the tools to the primary activities for each organization, the
310 representation of the toolchain with named tools for activities was sent to the organizations for verification. Data for the toolchains was adjusted and new tools added based on the responses. All but two case organizations (C13, C14) responded to the verification request.

3.5. Synthesis

315 Synthesis based on the research data commenced once the data had been coded and confirmed. Oral descriptions of the development process, process illustrations drawn by the respondents and the codified data of tool usage in different development stages were used to construct a representation of the development pipeline for each organization. The constructed development pipeline
320 representations and information about the toolchain allowed identifying pipeline sections and activities where work was done without tools. Qualitative analysis of the interviews was used to further analyze and describe the reasons behind the selection of the set of tools in an organization, their usefulness and the rationale for the current workflow.

Table 2: Metrics analyzed in this study.

Metric	Interview Questions (Appendix A)
Actual releasable software cycle	2.2, 2.3, 7.1, 7.2 and 7.3
Actual release cycle	2.2 and 2.3

325 The deployment and delivery frequencies of the organizations were analyzed by coding the verbal responses of the frequency-related questions in the interviews as numbers. These numbers represented the deployment and delivery capability of the organizations. The deployment and delivery capability of the organizations was compared against the amount of tool-aided phases in each
330 case for which the data was extracted from the toolchain representation. The definition for tool usage was binary: either the organization had tooling support for the activity or it did not.

Two frequency metrics were extracted from the interview data (Table 2):

- 335 • *Actual releasable software cycle* means an actual cycle during which the development organization produces an artifact that could be, in principle, released. However, factors out of control of the development organization can prevent realizing this release capability. There was no distinct question to acquire this metric, instead the metric was extracted from multiple answers. The interview protocol allowed such flexibility.
- 340 • *Actual release cycle* means how often the software is actually released. This takes into account other factors of the release.

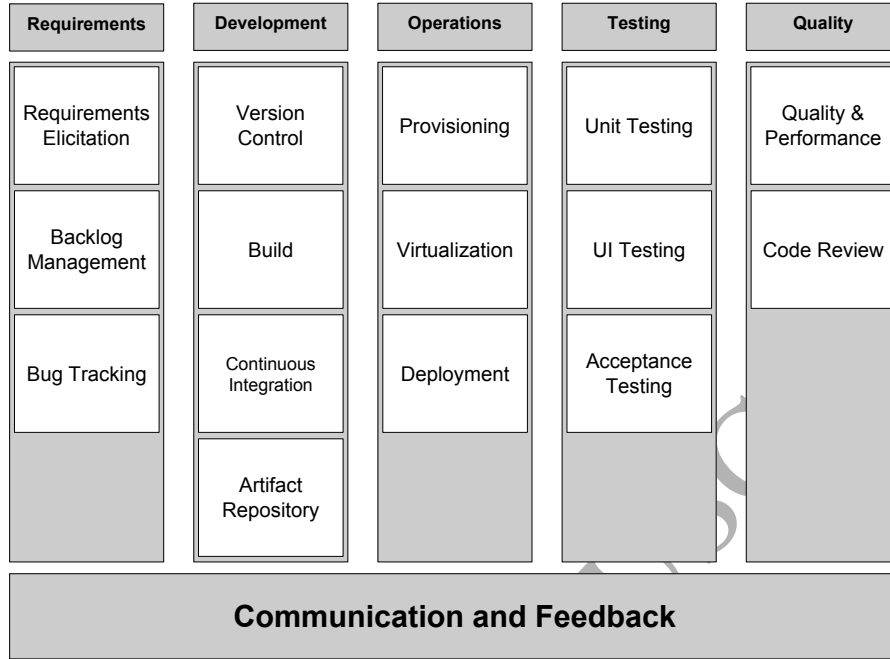


Figure 3: Elements of Modern Software Development Toolchain

Results yielded from counting the tool-aided phases were subject to quantitative analysis which allowed grouping of the organization based on their deployment and delivery capabilities, and the amount of tool aided phases. This process is further described in Section 6.

4. Modern Software Development Toolchain

In this section, we introduce a modern software development toolchain which was constructed based on the interviews conducted and the results of the thematic analysis in this study. Each element of the toolchain is described individually, accompanied by references from the literature. The main elements covered in the section include *requirements* as the first link in the toolchain, *development* as the core element for software development, *operations* as an element interfacing between various environments, *testing* as the element for verification and validation, *quality* as an additional element ensuring the non-functional quality properties, and finally the overarching *communications and feedback* which provides a common link between the elements.

A modern software development is backed up by a toolchain that consists of automated tools from code all the way to delivery of new software. The role of these tools is to ensure each stakeholder can develop, test and deploy the software when they need to and to make sure suitable feedback for from all stages of the development process is produced. Figure 3 illustrates the elements of modern software development that might benefit from tool support.

4.1. Requirements

Deploying software constantly, even several times a day, requires that software development teams have clear requirements for features to work with. Without a steady flow of requirements in the beginning of the deployment pipeline, the development team would not be able to keep up a constant pace. Regardless of the pace, keeping track of all the data associated with requirements is an essential part of software development. Here, *requirements elicitation* and *backlog management* tools assist in the process.

The classic user requirements workshop organized just once in the beginning of a project is no longer seen as the optimal solution due to the chance of miscommunicating requirements, and the rate of change in the requirements during the actual development. Thus, there is need for continuous feedback systems as detailed by Maalej et al. [27]. The modern feedback systems work both ways, pull and push. User requirements can be obtained not only by asking users directly through explicit pull mechanisms, such as workshops and interviews, but also by a push mechanism straight from the users, like feature and enhancement requests. The users can also be experimented on without their knowledge by implicit and tacit field experiments.

In modern-day software development, prioritizing work and knowing what needs to be done relies on maintaining a backlog, which is essentially a cumulative collection of work to be done. According to agile experts, backlogs should be ordered lists of well-defined work items. There can be several backlogs with items of varying granularity. For example, Scrum advocates two backlog artifacts: a product backlog, with feature-scale items, and a sprint backlog, with tasks that one person can do in a couple of hours. *Bug tracking* is also closely related to backlog management as a bug to fix is a specialized case of a development task for the developer – a work item among others. However, in some cases, bugs are also reported by customers or end-users, and a specialized tool may be needed to facilitate communication.

4.2. Development

Version Control Systems (VCS) exist to manage changes in various documents, such as source code and web pages, and other information that might change during the development of the software. A VCS stores information to retrieve any revision of the documents, so that the user may return to an earlier version of the modified document. Contemporary version control systems are usually distributed, as opposed to older centralized revision control systems. See Otte [28] for comparison of different VCS types.

Build systems are elementary for the development of software. Fundamentally, build systems construct deliverables based on build specification files written in a configuration language [29]. Build specifications can include instructions for compiling code, executing test cases, packaging project files and deploying new software releases [30]. Build tools which can interpret the specifications also help the developers in managing the internal and external dependencies of software, for instance with the help of proprietary mechanisms for resolving used external libraries [29].

One of the challenges with build systems is that when software systems grow in size and complexity, so do the build specification files, requiring much effort to maintain [29]. Build systems running large software projects can also face

resource shortages when a great number of build artifacts needs to be created over and over again, an issue which can be alleviated with the help of incremental builds or cloud-based parallel execution of tasks [30].

As an additional challenge to the size of the system, a software project might not only have internal dependencies to its own components but to external components developed by others as well. These can be libraries created by other teams within the organization or individual libraries, modules and packages developed elsewhere, containing features utilized by the project in question. When a multitude of such dependencies exist, it becomes increasingly difficult to manage the different versions of each dependency and the interdependencies between the dependencies themselves [31].

Automated builds bring in the principle of *continuous integration*, which in turn is a prerequisite for continuous delivery and continuous deployment. While CI can be implemented with an old computer and a rubber chicken [32], that is with modest investments and suitable practices using the chicken as a passable token, in modern software development continuous integration tools form a backbone for any continuous deployment pipeline. The task of continuous integration tools is to monitor version control systems for changes that consequently trigger the execution of the various steps configured on the continuous integration server [33]. The steps can include the compilation and packaging of code in order to build an identifiable software version, and execution of test suites with the aim to inform the developers whether their recent changes were successfully integrated or not.

Together with build systems, *artifact repositories* offer a solution to developers for managing dependencies and versioning different libraries. It could also be considered that artifact repositories help with the issue of incremental builds [30] since it is possible to store already compiled and tested, known to work, versions into the artifact repository.

4.3. Operations

Software can run on environments ranging from purely physical to purely virtual. *Provisioning*, *virtualization* and cloud environments make possible to set up instances of the software's running environment. Typically different environments, such as staging and production, are used in parallel during development. Those should however be such that the same build can be deployed in any of them. When it comes to continuous *deployment*, it should thus be relatively effortless to spin up new environment instances so that the build could pass from development to production without many obstacles or problems originating from the differences in execution environments.

Humble and Farley introduce common antipatterns related to delivering software. These are 1) Deploying software manually, 2) Deploying to a production-like environment only after completing development and 3) Manually configuring the production environments [17]. Instead, the aim should be in delivering software as automatically as possible. To avoid challenges related with changing configurations the development environments should also be as close as possible to the production. The same process should also be used for deploying to all environments.

In continuous delivery and deployment, the deployment infrastructure can be treated as code, and it may also be versioned. Ultimately, the system can

be reproduced wholly from the VCS. One approach for release management to maximize continuity and similarity of environments is blue-green deployment [17, 34]. In it, two identical versions of the production environment are maintained – a blue one and a green one. These two take turns in being the production. Each new deployment is done to the one not acting as production at the time. Moving to the new software version is done by switching the roles of the environments. The approach also makes a roll-back to the previous release possible and smooth, if something seems not to work in the new version.

4.4. Testing

Even though practitioners usually refer testing a separate activity, through test-driven development (TDD) and the introduction of continuous integration, contemporary testing is very close to the actual development activity. This is especially true with *unit testing*. Continuous integration tools with automated tests have to ensure that changes made in development do not introduce new bugs. The goal is to maintain a build that is tested against a thorough set of unit and component tests at each integration. Maintaining a more complete set of automated tests facilitates regression testing as the tests can re-executed in subsequent iterations. For the purpose of continuous deployment, it is still sensible to handle testing as an element of its own right, especially when it comes to acceptance and UI testing that are more difficult to fully automate.

User interface (UI) testing is a testing task where the graphical user interface is tested. A relatively small program may have a considerable amount of UI test cases and different sequence of tests may cause differing response from the software. Furthermore, a huge redesign of the UI may cause all user interface related tests to fail even though the actual functionality of the application may not have been changed at all. Some areas of UI testing, such as usability or user experience testing, fall close to the domain of quality assurance as they are not purely functional testing and may be hard to automate.

Acceptance testing is a special case of testing, where it is determined if the software system conforms to the requirements, either functional or non-functional ones. From the point of view of continuous deployment, acceptance tests should be run in an environment like the production. Humble and Farley [17] argue that acceptance testing should be automated and such that each build gets tested against acceptance tests provided it passes the earlier tests.

Traditionally, acceptance tests have required customer involvement in the execution of the tests. However, this is not typically possible if acceptance tests are run for each change to the software. Therefore, in this paper, we usually mean *automated* acceptance tests when we use the term "acceptance test". For automated acceptance tests, customer involvement is required only when constructing the automated test case, not when executing it.

4.5. Quality

Functional testing alone is not enough to ensure the quality of software products. However, not only continuous deployment, but agile practices in general, promote high software quality also on the code level and when it comes to non-functional requirements. Thus, additional care must be taken to take into account internal quality issues and other aspects of quality, such as performance and security.

One apparent way to assure *quality* is to measure certain metrics of the actual software system. For example, *performance* is usually difficult to assure without measuring it from the software execution. Thus, certain artificial load and stress tools may be used to ensure the functionality of software under even
 510 hardest circumstances.

Several aspects of the quality may be checked even without having the software system built at all. One way to achieve this is to have static program analysis tools, such as Lint, to check the code for a certain patterns, which may indicate bugs, vulnerable parts, suspicious code or just bad coding conventions. In certain domains, such as the medical software domain, their use is
 515 commonplace to improve the quality of safety-critical code.

Another way to improve code level quality is to have other human beings to review the code — even in the form of pair programming [3]. In the continuous deployment mindset, *code reviews* represent a way to introduce some human intervention to the otherwise automated pipeline. Some aspects of software quality may be hard to inspect automatically, so reviews can act as an additional safety guard. Code reviews are an old practice, but nowadays formal review sessions taking up to hours and covering large parts of code have fallen out of popularity. Instead, it is commonplace to integrate a code review tool to
 520 version control system so that every committed change may be reviewed by one or several persons. Persons may have various roles, for example certain people may have to give a permission for the commit to be integrated in the code base after they have reviewed it.

4.6. Communication and Feedback

Communication and collaboration are key elements in software development. Developers in software teams share knowledge through various mechanisms, which not only can help the team to build a social identity as a team but to also share task-related project knowledge between members [35]. Collaborative frameworks have use beyond the developer organization itself as software users
 530 can be integrated into software development with collaborative technologies [35], creating continuous push-and-pull feedback system [27]. A continuous deployment pipeline can be fully functional in a technical sense but without handling feedback efficiently from the developers and users alike, the development team might miss important cues in which direction to head next. Optimally, feedback
 540 from users and developers is directly associated with traceable software requirements and eventually the tasks which are put in the continuous deployment pipeline.

Feedback is an element that is crosscutting all stages of software development. One of the reasons behind the modern development toolchain is to make timely feedback reachable for all stakeholders. Timely feedback and short feedback cycles are a cornerstone of lean and agile. In continuous deployment, feedback
 545 is propagated from each integration.

5. Interview Results

In this section, we describe the tool usage in the case organizations. First,
 550 we overview the results and then we follow the elements described in Section 4 for the description of the results. Results for the general areas of requirements,

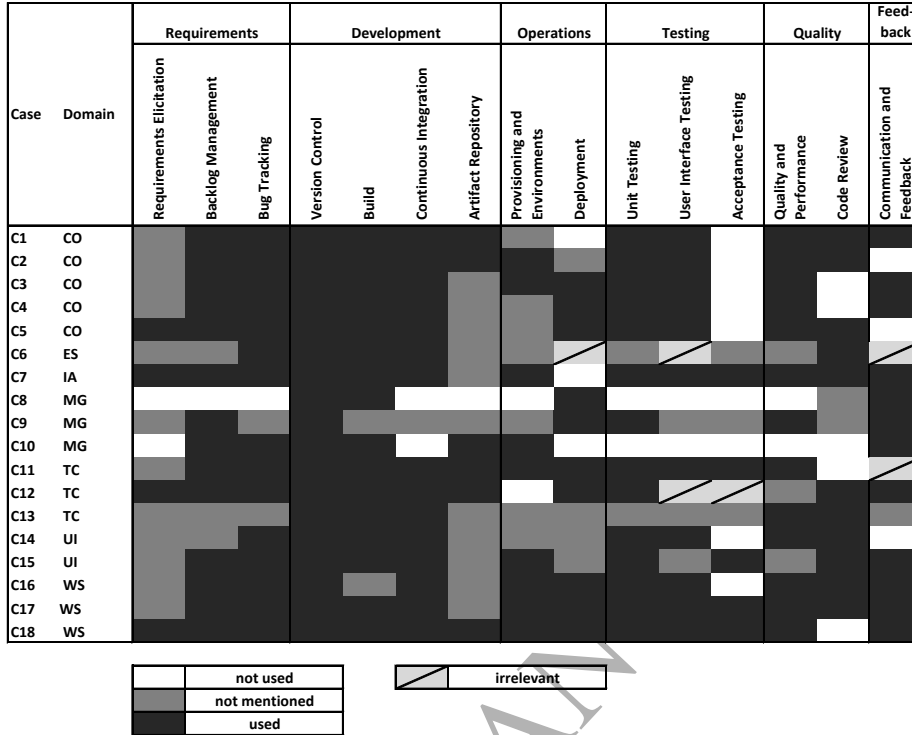


Figure 4: Summary of the tools used in the case development organizations.

Domains: **CO** = Consulting, **ES** = Embedded Systems, **IA** = Industrial Automation, **MG** = Mobile Games, **TC** = Telecom, **UI** = UI Framework, **WS** = Web Service

development, operations, testing, quality, and communication and feedback are each covered in turn.

5.1. Overview

The overview of the results for each software development stage is illustrated in Figure 4. In the figure, grouped by domain, tool support for every case has been plotted based on the information retrieved from the interviews. Rows in the figure represent the results of a single interview. Entries marked with black mean that a specific tool was **used** in the organization for the development stage and white means that any tool for this stage was **not used**. In certain application areas, some of the tools were considered **irrelevant** by the interviewees (e.g user interface testing for embedded systems): these entries are marked with light gray. When there was not enough information to determine whether an organization used a tool in an activity or not, we categorized the entries to the **not mentioned** category indicated by the dark gray color. The results were later confirmed from the case organizations, although confirmation was not received from two cases (C13, C14).

In total, results from 18 cases are shown in the figure. The results indicate that while handling requirements has shifted towards a more agile way of working by de-emphasizing the usage of formal requirements engineering tools

Table 3: An overview summarizing reported tool usage in different stages of development

Dev. Stage	Overview
Requirements	Backlog management tools were common and had replaced the rarely utilized specific tools for requirements engineering. Physical equipment like whiteboards can replace the need for backlog tools. Defects are handled with tools in most organizations.
Development	Version control practices had permeated all the organizations but branching strategies differed. Automated build systems for optimizing the build process were almost as common. Continuous integration systems were common except in the mobile games sector. Artifact repositories for storing precompiled libraries were not mentioned often.
Operations	Provisioning tools to configure and construct production-like environments were modestly used. Deployment tools had similar popularity although deployment was manual in many cases.
Testing	Programming language specific unit testing tools were commonplace. User interface testing was modestly tool assisted but hard to automate. Acceptance testing relied mostly on human judgment.
Quality	Besides separate testing procedures, quality was assured by various methods. Code quality was kept in check by occasionally used static analysis tools and code review protocols. Non-functional qualities were assured by tools for real-time monitoring especially in the web domain.
Communication and Feedback	Internal communication tools, sometimes associated with build and testing systems for rapid developer feedback, were used for development. The gap between developers and third parties was not often bridged with external communication tools. Customer data analytics were used sparingly.

and applying flexible management methods with ticketing boards, organizations have the first half of the toolchain quite well covered. Development is done through shared code repositories with good continuous integration support. Testing tools are used but user interface testing and acceptance testing are somewhat overlooked. Continuous deployment to production environments is not fully supported by existing tools in the organizations although certain deployment solutions are in use. Internal feedback tools are abundant but tools for customer feedback have less support. Table 3 shortly summarizes the observations of tool usage from each development stage. There seem to be certain domain-specific differences in tool utilization which are further discussed in the following sections.

A well-functioning continuous deployment pipeline requires that the output from each stage moves effortlessly to the next development stage. Comprehensive single-vendor software suites where development stages are interconnected by proprietary solutions were nonexistent. Instead, the organizations relied on best-of-breed tools, some of which had common interfaces that made them compatible with each other. When it comes to local workstations, developers were at times given leeway in selecting suitable development environments they felt most comfortable with as long as they followed the generic development process in the organization.

Version control systems and continuous integration servers were the integration hotspots that typically triggered workflows and tools in other development stages. In an ordinary situation when the degree of automation was high enough in an organization, the developer would commit work from the local workstation to a particular branch in a remote version control system which in turn triggered

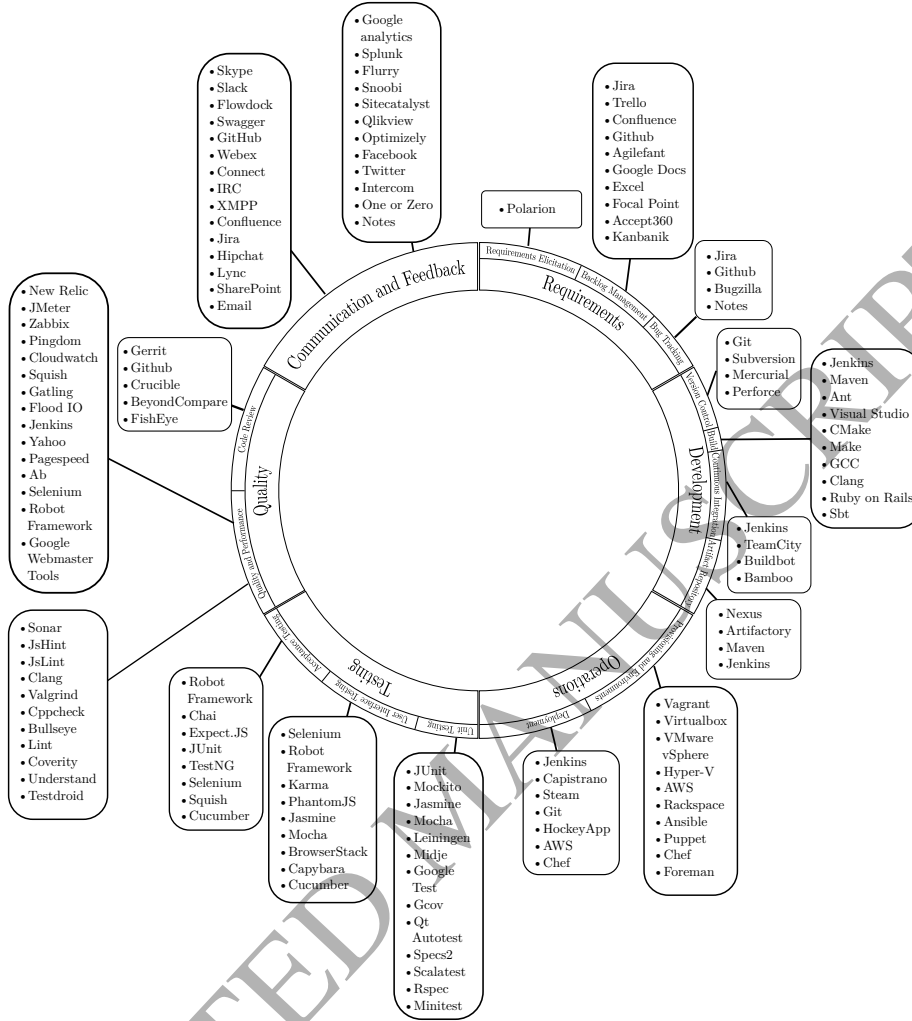


Figure 5: Overview of used tools in each development stage. For more information about the tools, see the tool appendix [36].

various testing activities through continuous integration servers, possibly provisioning the release to various environments closer to the production system. Similarly, tools for code review and version control were in some cases connected so that quality checks by peer developer reviews could be initiated after version control commits. Integration between development activities and tools was not entirely seamless and many tasks were still done manually, especially in the last stages prior to production releases, but at least in several cases work was being done to remove the bottlenecks and to reduce manual labor.

The concrete tools used by the organizations in each development stage reflected the development practices used in the organizations. While many of the tools can be considered independent of the development platforms and frameworks, the availability of tools and the current culture of tool usage in the field contributes to the choice of tools. In this article, we refer to the tools by name.

For more information about the tools, see the tool appendix [36].

610 Diversity of tools used in each development stage varied accordingly as depicted in Figure 5. Certain tools had universal applicability regardless of domain and context whereas others were more specific to platforms and frameworks. Tools for version control (Git), continuous integration (Jenkins) and code review (Gerrit) could be considered universal. Testing tools were more closely
615 associated with development platforms. Performance testing and monitoring tools such as New Relic had stronger support in the web domain where availability can be a greater concern.

5.2. Requirements

In a modern software development pipeline, the access to clear requirements
620 is ensured through *requirements elicitation*, *backlog management*, and *bug tracking*. The results show that requirements elicitation tools were used rarely and had been replaced by the more commonly used backlog management tools such as Jira. Tracking bugs with tools was common and alike for backlog management, Jira was the choice in many cases.

625 5.2.1. Requirements Elicitation

The results for our study indicate that organizations did not have many tools in use specifically for requirements elicitation. Closer to a hundred requirements elicitation tools have been identified to exist [37] but a specialized tool for requirements management and elicitation was used in only a single case where
630 they utilized Polarion Requirements.

The rest of the interviewed representatives, however, did not explicitly mention tools for requirements elicitation and managing requirements belonging to their existing toolchain. One organization, however, indicated that they use Atlassian Jira and Trello for requirements elicitation. Jira was also used for
635 backlog management, so it seems that this organization effectively transforms requirements into backlog items on the fly.

To illustrate the problems in the organizations, there is an example from one of the interviews. An interviewee mentioned that they were struggling to maintain a continuous flow of requirements for their development team. Their
640 requirements came in bursts so at times they were sitting idle, enjoying leisure time at their office, and then shortly after swarmed with new requirements when a person in the requirements engineering role had sufficient time to write them down.

Clearly, requirements engineering and elicitation tools were not commonly
645 used. That is not to say that the organizations would not have processes in place for requirements engineering and elicitation. Perhaps one of the reasons is that in the contemporary agile world, even large organizations are shifting towards more continuous forms of planning, where gradually prepared requirements through such instruments as user stories are seen to improve the development process [38]. Agile backlog management tools are thus seen more relevant
650 than traditional tools for requirement elicitation.

5.2.2. Backlog Management

The usual tool for backlog management is Jira (50 % of the cases), but Trello (17 %) and Confluence (11 %) also have a strong support. Jira has also been

655 documented as the most popular tool in previous studies [39], especially for bug tracking. In a sense, bugs can be considered specialized occurrences of backlog tasks.

Tracking issues and directing the flow of development in a project can also be done without the help of software tools. Whiteboards and other physical
660 tools were in some cases just as important as software in backlog management. Matter-of-factly, one interviewee commented that “the whiteboard was predominantly the most important tool in their project” as it could be used in tying up information from version control, backlog management, code review, bug tracking, acceptance testing and deployment. Thus, it was also a means of com-
665 munication as project members could use it to convey important information. It seems that a whiteboard is easy to use and gives transparency to information as it can be checked by just giving a quick glance.

5.2.3. Bug Tracking

Bug tracking is closely related to backlog management since a bug fix is a
670 certain type of a development task for the developer. However, in some cases, the bugs are also reported by the customer or the end-user, and a specialized tool may be needed to facilitate communication. Thus, it is not surprising that Jira is the most popular tool for the bug tracking as several organizations already utilized it as an internal tool for backlog management. Other tools,
675 also used for backlog management, were Trello and Github. Both of these was encountered once in the interviewed organizations. Three organizations used unnamed proprietary tools for bug tracking.

5.3. Development

Development work is supported in the pipeline with *version control*, *build*
680 *systems*, *continuous integration* and *artifact repositories*. Results for development suggest that the usage of version control tools is pervasive and Git has a strong foothold in the category. Build tools are almost as crucial to development as version control but the range of options for building is broader. Usage of continuous integration servers, mainly Jenkins, is frequent but there are exceptions
685 to the rule. Artifact repositories were not often mentioned.

5.3.1. Version Control

Almost all organizations used distributed version control systems, such as Mercurial and Git. Hybrid solutions and centralized systems were only seen in three organizations. It is interesting to note that even though the majority
690 of organizations use Git (almost every third), their practices using it vary. At least three fairly well-known branching models have been applied; “A successful Git branching model/Git Flow” [40], “Another Git branching model” [41] and “A rebase workflow for Git” [42]. In addition to the mentioned version control systems, some proprietary variations exist. Further research is needed to analyze
695 reasons and rationales behind different choices and their impact on the actual development process.

The existence of a VCS was ubiquitous to all organizations we interviewed. It was the only tool class to achieve this status. Thus, it underlines the need for this kind of a tool and its place as a commodity for a software developer. This is
700 interesting to note, as a decade ago there were reports of organizations who did

not use version control [43]. We can only speculate if the tools have achieved a certain degree of familiarity and maturity to be have such widespread use.

5.3.2. Build

Based on our interviews, organizations were using several different systems for building. Here, it is good to distinguish between build systems which run locally, and systems which run remotely on a server such as a continuous integration server where additional build steps can be executed, possibly triggered by changes in version control.

Around thirty percent of the respondents mentioned that they used Jenkins as their build system, which is also used as a continuous integration server. Java-based build systems Maven and Ant were almost as popular. Jenkins was mostly used as a remote build system that is used to launch the execution of a series of scripts containing actions for build steps. Although Jenkins utilizes build systems like Maven and Ant that can be run locally, in several cases build steps in Jenkins contained more complex workflows such as the orchestration and creation of virtual machines for testing and transferring content between machines. Build systems dealing with makefiles were used as well: Make and CMake were quite common choices although makefiles can contain overly complex structures due to their design [29].

Several interviewees said they used proprietary scripts as build systems while others commented that they used their integrated development environments for building. The choice for a build system can also be tied to the programming language. Certain systems can be built best with native tools for that environment. Furthermore, dynamic programming languages such as Ruby do not require compilation. Without having code to compile, the build steps in a Ruby on Rails application can include other actions such as precompiling Javascript assets used in web development or preparing databases and test fixtures for testing stages.

Given their elementary role, build systems and their smooth operation can be essential to continuous delivery and deployment. A badly working build system creates lag in the development process. An interviewee detailed that several years ago it took over two hours for the build system to complete building. They subsequently improved their system by buying new hardware, utilizing parallelism and incremental builds. Each of the improvements gradually improved their development capability by reducing build times and currently the build takes only three minutes to complete. That is quite a drastic change in performance.

It can be considered that build activities can generally take a long time to complete. Similarly to the previous case, a representative from another organization working in the embedded domain mentioned that build times for them range from ten minutes to an hour depending on the environment. He also said that roughly half of their working time is spent on waiting for the test runs to finish. An efficient build system which compiles the code fast, reuses already compiled artifacts and executes the tests in a reasonable time would likely be on the checklist for any organization aiming to do continuous deployment.

5.3.3. Continuous Integration

The interview results show that continuous integration servers are frequently used in organizations as part of their development pipeline. A vast majority of

the organizations mentioned that they used Jenkins as their continuous integration solution whereas other solutions such as TeamCity, Circle CI, Buildbot or Atlassian Bamboo were seldom in use. It seems that Jenkins has a firm foothold in the toolchain of organizations which might be due to its nature as an open source alternative to the other proprietary options.

The continuous integration workflow of the organizations matched the description of Meyer [33] quite well. Continuous integration environments had been set up so that processes on the continuous integration server were triggered by the changes in a version control system which was being monitored. Continuous integration servers were used to compile and build a software version with all the necessary dependencies which could then be put to test. Actual test execution was in some cases configured to run in external quality assurance environments where the continuous integration server pushed the freshly built software package to. However, pushing the latest version to different testing environments was not always automatic and required some manual work.

Feedback from the continuous integration servers was considered important, too. Certain interviewees regarded the continuous integration server as a safeguard for detecting abnormalities, catching the defects early on. Developers could observe the output of the continuous integration servers, and see whether their recent code changes passed the bar. Continuous integration servers may even show green status signifying success on the developers' screens or on the information radiators found in the physical space. An example developer space with two radiators from one of the cases is shown in Figure 6. The feedback loop back to the developer can be quite fast as a developer said that they can get the information from the continuous integration server in just a few minutes.

Passing the automated tests on a continuous integration server can be considered crucial when it comes to advancing to the next phase in the continuous deployment pipeline. There is little confidence in the software version if it does not pass the assigned tests it is supposed to pass. In cases where the build did not automatically progress to the next phase, respondents explained that they have to see that all tests pass before they can make the decision to trigger processes that promote the version in the end, i.e. accepting that the version in question is good to go.

Interestingly enough, three organizations reported not having any specific continuous integration environment. All three shared a common trait beside their usage of tools: they were small enterprises operating in the mobile games sector.

Why would a mobile game organization choose to omit automated builds and testing through continuous integration servers? An interviewee from a games organization stated that they did not have need for continuous integration since the developer could quickly, in less than a minute, run all the test locally without the help of external environments.

Still, mobile games – why not games in general – have properties which constrain the usefulness of automated testing which might not be present in other domains as such. In another games organization, the representative explained that there are only few things that could be automated. Testing of a game requires subjective understanding of the elements which eventually make the game fun to play. The same ideal was reflected in the opinions of the third organization: they had a minimal set of tools for testing and they preferred testing of their product on real users. Hence, under special circumstances, continuous

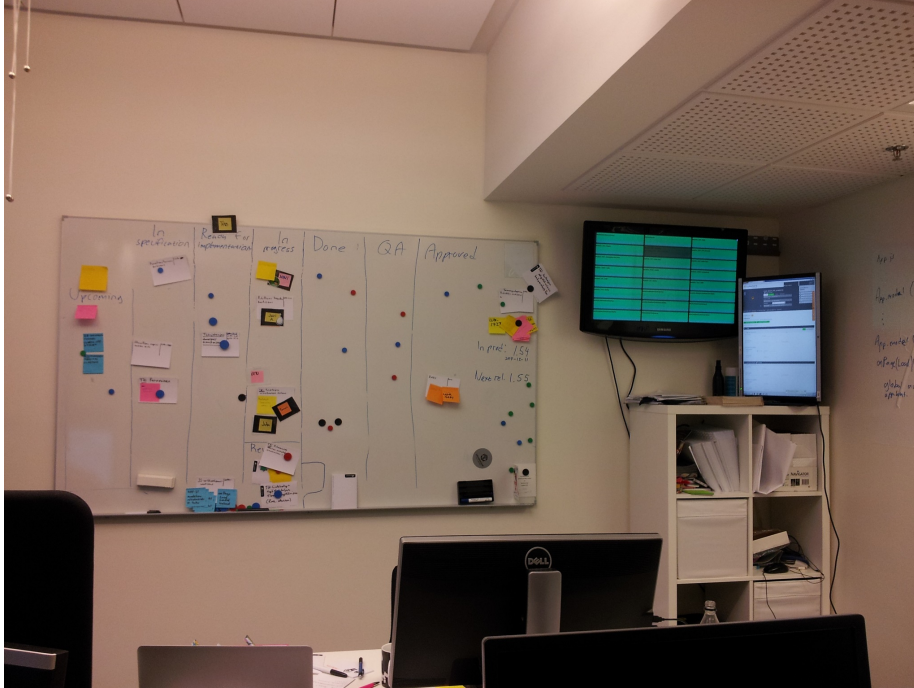


Figure 6: Radiators used in software developers' work spaces

integration tools are not always considered essential, especially so if circum-
 800 stantial factors prevent the fully streamlined flow of a continuous deployment
 pipeline.

5.3.4. Artifact Repository

Artifact repositories were not so commonly used in the organizations. It
 seems that declaring the dependencies in local project files and downloading
 805 the libraries straight from external, public, repositories was more common than
 actually using an artifact repository within the organization.

An internal artifact repository such as Artifactory was used to store compiled
 binaries in some cases. Others reported using Sonatype Nexus, and there were
 some custom solutions for artifact repositories, too.

810 5.4. Operations

Managing technical infrastructure and deploying software requires attention
 to be paid to *provisioning and environments* and *deployment* tools. Infrastruc-
 815 ture tools for providing environments were modestly used but organizations took
 advantage of virtualization technologies. Deployment tools for automating the
 last mile to production systems were uncommon.

5.4.1. Provisioning and Environments

Interviewees had a few shared concerns when talking about their infras-
 tructure. Replicating development environments was seen essential by several
 developers who wished to avoid mistakes originating from incompatibilities be-
 820 tween the local environments of developers and those used in later stages such

as testing and in production. For that purpose they mentioned to use a tool called Vagrant that can be used with virtualization technologies to clone exact environments from scratch during the build stages. This way, there should be no difference in environments. In one organization a separate product, Ansible, was used to orchestrate the creation and provisioning of environments. VMware and Virtualbox were popular choices for running virtual machines. For configuration management, the organizations preferred to use Chef and Puppet.

In fact, around half of the organizations used some kind of virtualization technologies as part of their workflow. If the products are being hosted in a public or private cloud, it could be considered essential to have tools available for creating, configuring and provisioning the remote servers. Otherwise this work would have to be done partly manually. The organizations reported to use common cloud service providers and their services in various ways.

In one organization they found it useful to first provision the new software versions to completely new virtual machines running in the cloud, waiting the servers to update while still running the old servers with the old version. Only when provisioning of enough machines was complete, the switch to the new version was finally made. By using the virtual machines this way, they did not have to keep their servers down for very long after an update. This is an example how having scalable external resources available can minimize the downtime and the negative environment-related effects of version updates.

5.4.2. Deployment

Deployment tools were not too common and the last mile of deployment involved varying amounts of manual work. Those who were the closest of being able to deploy automatically used a technique called blue-green deployments [17] where a new, identical environment to production called blue is set-up alongside the currently running green one. When the so called smoke tests pass in the blue environment, a switchover between the blue and green environments can be done with little downtime. Jenkins, the general purpose continuous integration server, was one of the tools mentioned to be used in this context. Jenkins was also used in other organizations where deployment was half-manual so that the build job to production or other environments could progress only once approved by chosen individuals. Automated deployment jobs supported by Jenkins were utilized to deliver tested builds from one staged development or testing environment to another, or even to production. Scheduled deployment jobs in Jenkins helped specifically when the deployment had to be done outside office hours.

Capistrano was a choice for some organizations that had deployment tools; this tool was used together with software written in Ruby although it could be used with other programming languages as well. While not strictly related to production environment deployment, one organization used a staging framework called HockeyApp to create a production-like environment in which they could test out their mobile applications before deploying them to production environment. Several organizations reported to have an in-house developed tool for deployment.

5.5. Testing

Testing divides into *unit testing*, *UI testing* and *acceptance testing*. While common, there were differences in the usage of unit testing tools and prac-

870 tices across industry fields. Difficulties in testing user interfaces showed in the responses although there were some tools used for the task. Automated acceptance testing tools were uncommon.

5.5.1. Unit Testing

875 Most organizations used some kind of unit testing framework which is in line with common development practices. All used unit testing frameworks were open source and depended on the technology stack of the organization. For example, organizations working mostly with Java used JUnit (seven cases), those working on C++ used Google Test or Qt Autotests (three cases) and organizations involved with web development stacks and Javascript used Jasmine or Mocha (two cases).

880 Unit testing was not mentioned or used in four cases. In two of the cases, specific tools for unit testing were not highlighted in the interviews although unit testing practices were discussed in general. In the other two cases from the mobile game sector, unit testing frameworks were explicitly stated not to exist and in one there were no plans to incorporate such frameworks. These 885 mobile gaming organizations considered unit testing challenging and not too valuable because games rely much on player interaction. Games are also heavily tested manually because graphical and interaction bugs are only detectable by the human eye. Therefore, bugs that unit tests would discover should anyway be discovered when manual testing is done.

890 5.5.2. UI Testing

Several interviewees, especially all from the mobile game domain, said that testing user interfaces is a hard task to automate. Thus, the lack of automated UI testing is one of the biggest challenges stopping the organizations from using a fully continuous deployment pipeline.

895 Even though these organizations had tools to aid them in UI testing, some additional manual testing was usually involved in full tests. One notable class of test tools include browser automation and cross-browser testing tools such as Browserstack, Selenium and PhantomJS.

5.5.3. Acceptance Testing

900 Acceptance testing is special testing where it is determined if the software system conforms to the requirements, either functional or non-functional ones. However, in the scope of continuous deployment, acceptance testing is usually seen as the last tests to decide if the system can be deployed. As such, it seems that the trust for tools to handle this kind of activity is low. Only five organi- 905 zations had automatic tools for acceptance testing. All other organizations had either team decisions, product owner decisions or manual testing as the means of acceptance testing. One organization used behavior-driven development tool, called Cucumber, to define acceptance tests.

5.6. Quality

910 Regarding quality there are overall issues with *quality and performance* as well as *code reviews* to consider. Assuring non-functional properties of products and systems was modestly tool-assisted with performance being the primary quality attribute being targeted. Code reviews assisted by the tools Gerrit or Github were a relatively common practice.

915 5.6.1. *Quality and Performance*

Static analysis tools for code quality were not very common in general, but in embedded domains, such as medical and telecommunications, their use was widespread. One interviewee indicated that – as the quality requirements are quite strict in this domain and the deployment process relatively slow –they
920 want to invest the effort to find out quality problems before deployment.

In web service domain, performance was the most important quality aspect. As weak performance may result in bad user experience or denial of service, it is natural that the organizations pay attention to this. New Relic was the most popular tool, but a wide variety of service provider tools and stress tools
925 were also used to ensure performance under heavy loads. These tools include, but were not limited to, Google Webmaster Tools, Google PageSpeed, JMeter, Gatling and Flood. Other group of tools were used to ensure network availability in cloud services. Zabbix, Pingdom and Cloudwatch are examples of such tools reported by the organizations.

Yet again, one interesting group was the mobile game organizations. As their product will be used on multiple platforms, there is a remarkable amount of testing involved in ensuring that the games will run as intended on these. On the other hand, the organizations are rather small, so they face a proportionally huge task whenever changes, which may be platform dependent, are made. All
935 three interviewed mobile organizations had solved this by using subcontracting in testing.

5.6.2. *Code Review*

In practice, tool aided code reviews were almost uniformly conducted via Github pull requests or using the Gerrit tool. Both ways seemed to share similar
940 popularity. Some organizations relied on other tools also, such as FishEye or BeyondCompare. Fully manual reviewing – usually by pair programming – was also used.

The practices vary greatly; some organizations enforce reviewing for all commits to the version control systems, some organizations do not see any additional
945 value for mandatory reviews. However, even those cases might review some parts of the code, if they consider it to be critical for the quality. The interviewees also emphasized the effect of reviews in making the code more easier to understand. This is in line with the findings of Siy and Votta [44].

5.7. *Communication and Feedback*

Based on our interview results, instant messaging tools are quite widespread: Skype, Internet Relay Chat (IRC), XMPP clients, WebEx, Hipchat and Lync were all used for instant messaging. Two organizations used a special purpose
950 instant messaging application – Flowdock – for developer communications. This tool is designed for software development and it has hooks for other development and testing tools. For example, one of these organizations used Flowdock to communicate continuous integration server test failures to developers. Another web development organization used a tool called Slack for similar purposes.
955

E-mail was also used for communication and for notifications in several development stages. A traditional face-to-face discussion was seen as an important
960 way of conveying a message, too. Tools do not always have to be electronic: one interviewee proclaimed later that a whiteboard and a marker are their most important communication tools they use on a daily basis.

Besides managing internal communication channels, the developers have to deal with external users and their feedback about a software system. Feedback from the users can originate from explicit push communication channels [27] that allow users to explicitly express their wishes for the system. Our interview data suggests that organizations did not have many customer-facing explicit push channels: respondents said that they mainly answered to e-mails, phone calls or messages from contact forms. In one case it was mentioned that a Helpdesk service called One or Zero was used for client communication. In yet another interview it came up that an organization had previously been using a tool called Intercom for handling requests but this real-time system was too taxing for the developers to use in the end as they had to be constantly on call.

When developers take initiative and actively seek information from users, the communication mechanism can be referred to as pull communication [27]. These can be either explicit methods where developers and users collaborate such as typical workshops or interviews, or more implicit where product or service usage data is analyzed. There was some evidence of pull mechanisms and tools being used in our case organizations. Organizations mentioned that they do surveys or personally meet with their users to acquire feedback. Tools for implicit pull communication had limited support in web development as organizations were tracking their users with Google Analytics, Site Catalyst, Flurry or Snoobi and using supporting tools to visualize and understand data gathered from users.

6. Comparing Tooling and Deployment Capability

Next, we focus on studying the connection between tool usage and deployment speed. From the interviews of each case we have collected data about the time it takes to have a minimum deployable software, and how often actual release is done. These metrics respectively correspond to the metrics of *actual releasable software cycle* and *actual release cycle* mentioned in the study synthesis description of Section 3.5. Based on this information, we created a visualization which shows the differences between the release cycle and deployment capability of different cases. We did not have enough information about deployment times for one of the cases (case C17) so it is left out from the observations.

In Figure 7 the cases are set to timeline by the release time. In the visualization, the deployment potential, meaning the time to build a minimum deployable software, is visualized with black and time spent to actual release is visualized with grey. From the figure it can be observed that the longest release cycle is 1.5 years (case C5) where the shortest cycle is 20 minutes (case C16). We can also see that in many cases the deployment capability is much less than the actual length of the release cycle. For example in case C5 the deployment capability is one month but the release is done only once per 1.5 years. This shows well that the release time is not connected to deployment capability which is the case for many of the cases presented.

As the release time itself is not a good metric in determining how capable the organizations are in deployment, we made another visualization that discards the release time and shows only the time to minimum deployable software. This visualization is shown in Figure 8.

In Figure 8 the time unit is now days whereas it was months in the Figure 7. From the figure we can observe that case C13 has the weakest deployment

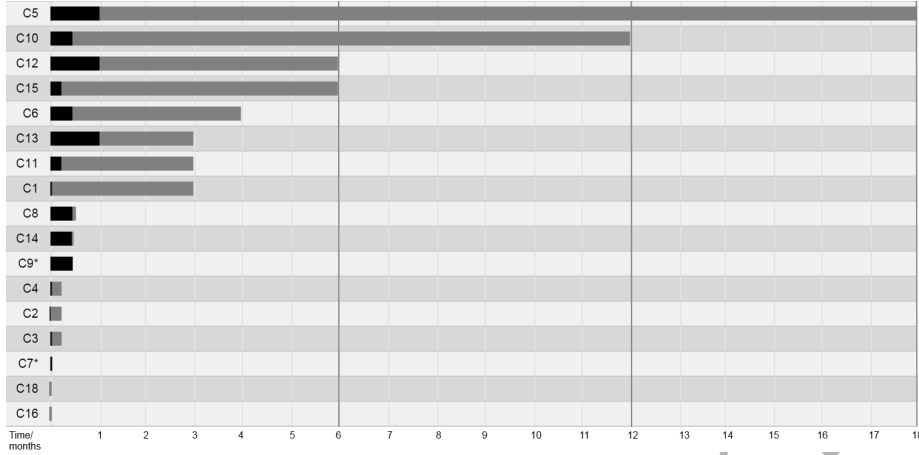


Figure 7: Release time of cases presented on timeline. Time unit in the figure is months. Black color presents minimum time to deployable software (deployment capability) and grey presents the actual release time.

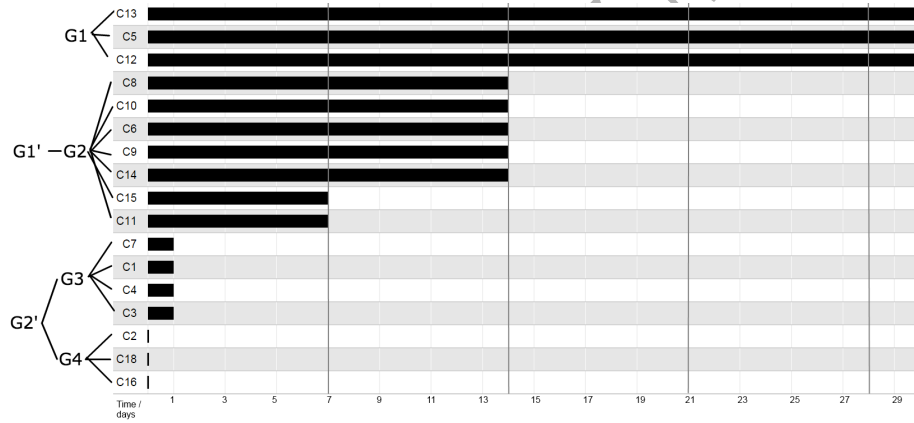


Figure 8: Deployment capability of cases presented on timeline. Time unit in the figure is days.

1010 capability, being 4 weeks, where the case C16 has the best deployment capability,
1015 being 20 minutes.

6.1. Case Groups

We can divide the cases into four different groups based on the deployment capability. In the first group, containing the slowest cases, the deployment capability is approximately one month. This group (G1) includes the cases C13, C5 and C12. The second group (G2) includes cases C8, C10, C6, C9, C14, C15 and C11. In this group, the deployment capability is from a week to two weeks. The third group (G3) includes cases which deployment capability is around one day. The cases in the group three are C7, C1, C4 and C3. The last group (G4) contains cases C2, C18 and C16 which deployment capability is less than an hour.

Case Domain		Requirements			Development				Operations		Testing			Quality		Feed-back	Group
		Requirements Elicitation	Backlog Management	Bug Tracking	Version Control	Build	Continuous Integration	Artifact Repository	Provisioning and Environments	Deployment	Unit Testing	User Interface Testing	Acceptance Testing	Quality and Performance	Code Review	Communication and Feedback	
C13	TC																G1
C5	CO																
C12	TC																
C8	MG																G2
C10	MG																
C6	E5																
C9	MG																
C14	UI																
C15	UI																G3
C11	TC																
C7	IA																
C1	CO																G4
C4	CO																
C3	CO																
C2	CO																G4
C18	WS																
C16	WS																

not used

not mentioned

used

irrelevant

Figure 9: Toolchain matrix presented in Figure 4 reordered by deployment capability.

Domains: **CO** = Consulting, **ES** = Embedded Systems, **IA** = Industrial Automation, **MG** = Mobile Games, **TC** = Telecom, **UI** = UI Framework, **WS** = Web Service

Groups: Cases are grouped by the deployment capability. **G1'** = slow deployment capability, **G2'** = fast deployment capability

To see if there is any possible connection in tooling and deployment capability we constructed a reordered version of the toolchain matrix presented in Figure 4. We ordered the matrix based on the deployment capability (to the same order as in Figure 8) and we divided the matrix into four groups based on the categories presented above. The reordered matrix is presented in Figure 9. From the toolchain matrix we tried to find similarities within the groups and also compared the groups to see if the groups with better deployment capability have more tooling in use than the groups that have weaker deployment capability.

Looking at the reordered toolchain matrix, we can notice that for the development part of the pipeline, except artifact repository, tooling is present for most of the cases. However, we also observe that the cases with the fewest confirmed tools belong to groups G1 and G2 which are the slowest groups in terms of deployment capability.

We also calculated the amounts of tool aided phases and phases where no tools are used in the toolchain for the organizations. In Table 4, the medians and arithmetic means for both types of phases are presented for the organization groups. These numbers act as indicators for the level of tooling in a certain group. We can see from Table 4 that the means and medians of groups G3 and G4 are close to each other. However, the G1 group has a median similar to G3 and G4, but a mean similar to G2. This observation refers to that the three cases in group G1 do not have high similarity in level of tooling. If we look at the individual cases in G1 we can observe that in cases C5 and C12 the use of tooling is overall in the same level as in the cases in group G3. However, the

use of tooling in case C13 is closer to cases in G2 since in the case of C13, tools have been mentioned only for five phases, being closer to the mean of G2 than to G1.

Based on the low similarity between cases in group G1, we discarded this group from further observations. Now group G2 can be renamed into group G1'. Furthermore, groups G3 and G4 can be combined into a new group named G2' based on the similarity. The means and medians for these new combined groups are also presented in the Table 4. Based on the data we can state that there is clearly more tool support in the group G2' which also have overall better deployment capability than group G1'.

Table 4: **Level of tooling between groups.** The level of tooling is calculated by counting the number of phases which are tool aided and the number of phases which are not tool aided for each case. The level of tooling for a group is then calculated by arithmetic mean from the level of tooling of the cases in the group, also medians are presented.

Group	mean and median amount of phases			Group	mean and median amount of phases		
G1	no tool used	6	4				
	tool aided	9	11				
G2	no tool used	7.29	7	G1'	no tool used	7.29	7
	tool aided	7.71	8		tool aided	7.71	8
G3	no tool used	3.75	4		no tool used	3.43	4
	tool aided	11.25	11	G2'			
G4	no tool used	3	4		tool aided	11.57	11
	tool aided	12	11				

When comparing the groups G1' and G2' we can find differences in tooling in many areas. In group G1' four cases out of seven uses unit test tools where in group G2' all cases use unit test tools. User interface testing is tool aided for all cases in G2' where in group G1' only two cases have stated to use tools for user interface testing. Differences can also be found from backlog management and quality and performance testing. In group G2' all cases have stated that they use tooling for backlog management where in group G1' four cases out of seven have tooling for backlog management. For quality and performance testing tooling is used in three cases in group G1'. Conversely, in group G2' all use tooling for quality and performance testing. For both groups, acceptance testing and code review were the areas where least tooling were used.

The observations refer to possible relation between tooling and deployment capability and seconds the statement that firm automated tool support is needed to be able to deploy fast. However, in the original group division group G1, which contains the three cases with the weakest deployment capability in terms of time, the use of tooling was somewhat in the same level as in G2' (the group with the best deployment capability). Based on this observation we can state that the degree of tooling is not a sole guarantee of fast deployment capability.

7. Discussion

In this section, the research questions of the study are answered and compared to related work. Limitations to the validity of the study are also discussed

and possible directions for future work explored towards the end of the section.

7.1. RQ1: Which toolchains are used to support software delivery?

Several observations can be made from the interview data. Looking at the interview results in Figure 4, some of the tool categories are used in almost all organizations. These *commodity tools* include *version control*, *backlog management*, *build*, *continuous integration*, *unit testing* and *bug tracking*. They are used in over two thirds (12 cases) of the development organizations. These tools can be seen as an industry standard, and almost every software development project should incorporate them in the development activities. However, at least with unit testing there seems to be some domain-related controversy. In the mobile game industry, unit testing is mentioned in only one organization which indicates that it is not a necessary component in every domain.

Some of the tool categories are not used as much. These *modestly used tools* are *provisioning and environments*, *UI testing*, *performance testing*, *code review*, *deployment*, and *customer feedback*. They are used in a third or two thirds of the development organizations (from six to twelve cases). It might be that some of these categories are not so useful in every context and domain. For example, UI testing is naturally unnecessary if the software does not have a UI. In addition, some categories can be performed well without any software tooling, such as code reviews and communication.

Some tools are used only in a third or fewer (fewer than six cases) of the development organizations. These *uncommon tools* include *requirements elicitation*, *artifact repository*, *provisioning and environments*, *testing of quality attributes* and *acceptance testing*. Again, requirements elicitation, for instance, can be done without tooling. Likewise, many organizations stated that acceptance testing is done manually. However, automating IT infrastructure, acceptance tests and having an artifact repository are considered to be important parts of continuous delivery [17].

Meyer [33] listed required tools for continuous integration. He mentions that use of version control, build, continuous integration, deployment and monitoring tools are necessary for the practice of continuous integration. In our cases, tools other than deployment and monitoring tools are used in the majority of the organizations. Deployment and monitoring (feedback category) tools are also used in over half of the development organizations (10 cases for deployment, 12 cases for feedback), but not in as many organizations as the other tools mentioned by Meyer. Thus, overall the case organizations seem to have comprehensive tooling for continuous integration, but the later parts of the continuous deployment pipeline are not covered in all organizations.

Based on the domain of the cases, it seems that mobile gaming industry does not rely on tooling as much as other domains. This might be because mobile games rely on heavy manual testing and games are typically so small in size as applications that it is feasible to test them manually.

When looking at the cases when interviewees have explicitly said *not* using a tool category, acceptance testing was most often mentioned as a missing component from the toolchain (8 out of 18 cases). This implies that automated acceptance tests are still quite rare and thus, continuous deployment is not feasible for most of the organizations.

7.2. RQ2: What are the reasons behind the gaps in the toolchains?

Based on the interviews of organization representatives, a number of gaps were identified in the toolchains of the organizations, and as noted, some of the tools for specific software engineering activities were more prevalent than others. Several reasons for the absence of tools in these areas surfaced in the analysis.

When a specific tool is not being used for an activity as part of the toolchain in an organization, it does not necessarily mean that the activity is completely disregarded. For instance, several of the organizations that were not using unit testing tools still relied on external testing services where they sent their products to be checked. While conserving internal resources, outsourcing testing might cause a notable delay in the flow of the continuous deployment pipeline. An activity can also be performed completely without tools. Code reviews are examples of activities where a tool can be helpful but not entirely necessary as peer reviews can be made more or less manually.

Since parts of the continuous deployment pipeline are interconnected, the tools have contingencies and are partly dependent on the existence of other tools. Those organizations that did not have unit testing tools also did not have user interface testing tools or continuous integration servers running. If there are no tests to execute, having a continuous integration server might not be worthwhile.

There might also be a gap in the toolchain of an organization because the tool is not seen as relevant. If an organization is mostly operating in the embedded system domain, it does not have a strong incentive to use tools for user interface testing. In this sense, a gap cannot always be seen as a flaw in the continuous deployment pipeline of a specific organization: the context and domain play an important role as to which tools are seen integral and which are not.

Widespread usage of a tool might be hindered by insufficient knowledge about the tool and the value it might provide to the organization. For instance, artifact repositories were only confirmed to be used by a handful of organizations and perhaps not many had seen the benefits of incremental builds or proper internal management and versioning of in-house libraries. Continuous feedback systems shared a similar fate to artifact repositories in a way. There were numerous internal communication tools used but far fewer were utilized to interact directly with the customers and end users or used implicitly to observe user behavior. Constant feedback can put a strain on the developers and thus keeping feedback loops closed can be a conscious choice.

Concerning the final push of the software version and deployment of the application to the production environment, organizations did seem to have a certain number of associated tools but the workflow was not fully automated. Deployments could in some cases be done with one-button pushes but deployment was preceded by manual verification and validation activities in a vast number of the cases. It is plausible that organizations might not see fully automated deployment tools attractive due to their manual activities in this stage of development. The various manual steps and checks performed before deployment might also explain why the adoption of acceptance testing tools was low: the organizations were somewhat unaccustomed to capture user scenarios as acceptance tests and were faced with the fact that user acceptance had to be obtained through more conventional means.

7.3. RQ3: What possible implications do toolchains have on the speed of software delivery?

Connection between toolchains and speed of software deployment time was studied based on the data gathered from the interviews. We noticed that the actual deployment time did not have much relevance to deployment capability which we defined as the minimum time to deployable software. We also observed that for the fastest cases the deployment capability and also the deployment time can be measured in minutes whereas for the slowest cases the deployment capability is over a month and the actual deployment time can even be well beyond a year.

As the focus was to study if there is connection between tooling and deployment capability, we divided the cases into groups based on their deployment capabilities and reordered the toolchain matrix presented in Figure 4 to match the order of cases in deployment capability timeline. The reordered toolchain matrix is presented in Figure 9. We observed that the toolchain is more complete in cases that can deploy every day or more often. Based on this observation we state that the toolchain should be as complete as possible to enable daily deployments. However, we also observed that even a good toolchain is not a guarantee for fast deployment. In some cases the deployment was done seldom but the toolchain was on the same level with the cases that deploy fast. In addition, cases in group G2 lacked many tools, but the organizations were still able to deploy every two weeks.

7.4. Future Work

In this section, we propose future research topics based on our observations. One of the research discoveries was that organizations tend to have a better capability to deliver software than they actually do in practice. This raises the question why there is such a great difference between the capability to deliver and the actual release cycle? Domain differences explain this to some extent, as in some domains customers are more reluctant to acquire new releases, but there could be other factors at play here. If the delivery capability of an organization is a lot shorter than the actual release cycle, does it still make sense to do rapid and small releases internally? For the organizations which have already set a steady and relatively rapid pace, what are the benefits of having daily releases as opposed to bi-weekly releases?

In this study, we asked the interviewees to describe their development process and name the tools they used while developing software but we did not ask them to rate the tools for usefulness. Which tools in which development stages are the most critical when it comes to rapid delivery and what tools do developers themselves find the *most* useful? Are the same tools needed if an organization only strives for a bi-weekly release cycle rather than a daily release cycle?

Besides seemingly having an effect on the capability to release software faster, toolchains could also have an impact, positive or negative, on product or process qualities such as defect rates. A strong support for test automation tools could affect perceived product failures as could tools in other development stages like operations and deployment; reduced manual configuration could potentially result in fewer mistakes being made. An increased reliance on tools and automation could mean, however, that certain types of bugs would go unnoticed if less human attention is paid to each release. The overall relationship between

toolchains and product qualities remain somewhat unclear and this could be yet another avenue for future research.

7.5. Threats to Validity

Case studies may have a number of threats concerning their validity. These threats can be classified into four themes: construct validity, internal validity, external validity and reliability [45]. In this chapter, we will address each of these four themes with respect to the results of our study. In addition, we address threats to validity concerning open-ended interviews as a data collection method.

7.5.1. Construct Validity

Construct validity refers to how well operational measures represent what researchers intended them to represent in the study in question. For an interview study such as ours, this means ascertaining the extent to which interview questions were interpreted the same way by both interviewers and interviewees. Considering toolchains, we find there is little room to interpret direct questions regarding their use differently. In addition, initial results from our interviews were sent to interviewees for inspection and approval, further strengthening the construct validity. Construct validity is to an extent threatened by the ability of the interviewees to recall their development process and related tools as other sources of evidence were not used besides the interviews. Having the interviewees illustrating their development process by drawing and asking them to add the tools to the depicted phases later reduces this threat to construct validity.

Considering frequency metrics, there is a certain threat to construct validity. Interview responses indicated that constructs like release, deployment and delivery might not be interpreted in the same way by all stakeholders. In order to mitigate the threat, the metrics were defined more precisely after the interviews and the interview responses were checked to match the definitions.

7.5.2. Internal Validity

Internal validity concerns studies in which causal relationships are examined. In particular, internal validity concerns efforts made to ensure that possible confounding factors are identified and alleviated. Threats to internal validity in the study arise from the implications of the amount of tool-aided software development phases in the organizations.

In the analysis and discussion of data it is implied that the degree of tool-aided phases has an effect on the release frequency of the organizations. The possibility of rival explanations for the speed of delivery cannot be ruled out as there are confounding factors which might affect the internal and external release cycles of an organization. The pattern-matching logic held in most cases, but not all, and confounding factors such as the domain of the organizations could already be identified. No strong causal claims are made due to the descriptive nature of the study but the implied validity threat is acknowledged.

7.5.3. External Validity

External validity concerns the extent to which results of a case study can be generalized. In most cases, results of a case study cannot be wholly generalized [23]. The multiple-case design and the replication logic with cross-case analysis increases the external validity of this study. Limited generalization for

similar contexts may be possible although there were not many cases for each domain, which poses a threat to external validity. While not guaranteed, a practitioner operating in a particular domain may gain insight to the development of their toolchain by comparing the study context and their own environment, and applying the findings.

7.5.4. Reliability

Reliability concerns the extent to which results are dependent on particular researchers. Ideally, a study should be able to be repeated by other researchers, achieving the same results [45]. We cannot guarantee that the coding of data in our thematic analysis is entirely void of bias; the reliability of any thematic analysis is inherently subject to differences in interpretations by researchers [46], although we mitigated this bias by sending the results back to the organizations for verification. The confidence in the results of the thematic analysis is increased by the amount of researchers involved in the study as the results and findings were cross-checked by multiple researchers.

8. Conclusions

Continuous deployment requires seamless collaboration between people working on various aspects of software engineering and a streamlined product flow from one stage to another. Using a sufficient amount of tools can help in achieving a state of continuous deployment but it is not always straightforward to select the right tool for the job.

The results of the interview study carried out in Finnish software development organizations reveal the prominence of tools in different development stages and that some tools can be considered more important in terms of continuous deployment than others. However, the relative importance of a tool is specific to an organization and the domain the organization operates in as the need for the speed of deployment can differ.

While there are activities, such as code review, that can be done without specialized tools, having proper tool support is essential in certain stages of development. Version control systems are so prevalent in the industry that they were found in all of the interviewed organizations. Although having somewhat lower support overall, tools in the areas of backlog management, building and testing had high adoption rates in the organizations as well. Support for other tools varied more and the results seem to indicate that the completeness of the whole toolchain had an effect on the delivery and deployment capabilities of the organizations but there were exceptions to the rule.

All organizations might not even be looking to fully achieve the state of continuous deployment which has a direct impact on their toolchain. Without apparent need to deliver software faster to the end users, there is no subsequent need for tools in the toolchain that enable continuous deployment.

Automating stages and the application of tools especially in the later stages of development, including system testing and deployment, might not be restricted solely by the availability of appropriate tools. Prevailing company culture and human nature can increase reluctance in adopting tools as there seems to be a tendency to rely on manual inspection rather than automated tests to spot latent errors and to verify the readiness of a release. Good reasons for

retaining control over releases exist but practitioners should be aware that the reasons might as well be cultural or related to having adequate trust on the automated toolchain.

It can be concluded that a well-established tool infrastructure supports shipping software releases continuously. Tools ranging from facilitating management of requirements, easing daily development activities, ensuring quality, speeding deployment, to handling rapid user feedback all may improve the delivery cycle. Organizations employing such tools in practice might just get the extra boost to development that would allow them to shorten release cycles and speed the delivery of new software to the ever-evolving markets.

Acknowledgments

We would like to thank the organizations for describing their development processes and the participants to the interviews for their time and effort. This article was supported by TEKES [47] as part of the N4S Program [25] of DIGILE (Finnish Strategic Center for Science, Technology and Innovation in the field of ICT and digital business).

References

- [1] D. J. Anderson, Agile Management for Software Engineering: Applying the theory of constraints for business results, Prentice Hall Professional, 2003.
- [2] K. Schwaber, Scrum development process, in: Business Object Design and Implementation, Springer, 1997, pp. 117–134.
- [3] K. Beck, Extreme programming explained: embrace change, Addison-Wesley Professional, 2000.
- [4] K. Beck, M. Beedle, A. v. Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, D. Thomas, Manifesto for Agile Software Development, retrieved: August 2016 (2001). URL <http://agilemanifesto.org>
- [5] J. Humble, The case for continuous delivery, <http://www.thoughtworks.com/insights/blog/case-continuous-delivery>, retrieved: August 2016.
- [6] T. Fitz, Continuous deployment, retrieved: November 2015 (2009). URL <http://timothyfitz.com/2009/02/08/continuous-deployment/>
- [7] M. Leppänen, T. Kilamo, T. Mikkonen, Towards post-agile development practices through productized development infrastructure, in: Rapid Continuous Software Engineering (RCoSE), 2015 IEEE/ACM 2nd International Workshop on, 2015, pp. 34–40.
- [8] Y. Monden, Toyota production system, Journal of the Operational Research Society 46 (5) (1995) 669–670.
- [9] M. Poppendieck, T. Poppendieck, Lean Software Development: an agile toolkit, Addison-Wesley Professional, 2003.

- [10] B. Fitzgerald, K.-J. Stol, Continuous software engineering and beyond: Trends and challenges, in: Proceedings of the 1st International Workshop on Rapid Continuous Software Engineering, RCoSE 2014, ACM, New York, NY, USA, 2014, pp. 1–9.
- 1355 [11] P. Debois, Devops: A software revolution in the making, The Journal of Information Technology Management 24 (8) (2001) 3–5.
- [12] L. Bass, I. Weber, L. Zhu, DevOps: A Software Architect's Perspective, Addison-Wesley Professional, 2015.
- 1360 [13] M. Fowler, M. Foemmel, Continuous integration, <http://www.martinfowler.com/articles/continuousIntegration.html>, 2005, retrieved: August 2016.
- [14] M. Fowler, ContinuousDelivery, retrieved: August 2016 (2013). URL <http://martinfowler.com/bliki/ContinuousDelivery.html>
- 1365 [15] M. Poppendieck, T. Poppendieck, Lean Software Development: An Agile Toolkit, Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2003.
- [16] S. Neely, S. Stolt, Continuous delivery? easy! just change everything (well, maybe it is not that easy)., in: AGILE, IEEE Computer Society, 2013, pp. 121–128.
- 1370 [17] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation, 1st Edition, Addison-Wesley Professional, 2010.
- [18] D. Ståhl, J. Bosch, Experienced benefits of continuous integration in industry software product development: A case study, in: IASTED Multiconferences - Proceedings of the IASTED International Conference on Software Engineering, SE 2013, ACTAPRESS, 2013, pp. 736–743.
- 1375 [19] D. Ståhl, J. Bosch, Modeling continuous integration practice differences in industry software development, J. Syst. Softw. 87 (2014) 48–59.
- [20] D. Ståhl, J. Bosch, Automated software integration flows in industry: A multiple-case study, in: Companion Proceedings of the 36th International Conference on Software Engineering, ICSE Companion 2014, ACM, New York, NY, USA, 2014, pp. 54–63.
- 1380 [21] M. Leppänen, S. Mäkinen, M. Pagels, V.-P. Eloranta, J. Itkonen, M. V. Mäntylä, T. Mannistö, The highways and country roads to continuous deployment, Software, IEEE 32 (2) (2015) 64–72.
- 1385 [22] G. G. Claps, R. B. Svensson, A. Aurum, On the journey to continuous deployment: Technical and social challenges along the way, Information and Software Technology 57 (0) (2015) 21 – 31.
- 1390 [23] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical software engineering 14 (2) (2009) 131–164.

- [24] C. Wohlin, A. Aurum, Towards a decision-making structure for selecting a research design in empirical software engineering, *Empirical Software Engineering* (2014) 1–29.
- 1395 [25] Digile N4S, retrieved: August 2016 (11 2015).
URL <http://www.n4s.fi/en>
- [26] D. Cruzes, T. Dyba, Recommended steps for thematic synthesis in software engineering, in: *International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2011, pp. 275–284.
- 1400 [27] W. Maalej, H.-J. Happel, A. Rashid, When users become collaborators: Towards continuous and context-aware user input, in: *Proceedings of the 24th ACM SIGPLAN Conference Companion on Object Oriented Programming Systems Languages and Applications, OOPSLA '09*, ACM, New York, NY, USA, 2009, pp. 981–990.
- 1405 [28] S. Otte, Version control systems, proseminar Technische Informatik, Computer Systems and Telematics Institute of Computer Science Freie Universität Berlin, Germany (2009).
- [29] S. McIntosh, B. Adams, A. E. Hassan, The evolution of java build systems, *Empirical Software Engineering* 17 (4-5) (2012) 578–608.
- 1410 [30] C. Prasad, W. Schulte, Taking control of your engineering tools, *Computer* 46 (11) (2013) 63–66.
- [31] J. Fischer, R. Majumdar, S. Esmailsabzali, Engage: A deployment management system, *SIGPLAN Not.* 47 (6) (2012) 263–274.
- [32] J. Shore, Continuous integration on a dollar a day, retrieved: August 2016 (2006).
1415 URL <http://www.jamesshore.com/Blog/Continuous-Integration-on-a-Dollar-a-Day.html>
- [33] M. Meyer, Continuous integration and its tools, *IEEE Software* 31 (3) (2014) 14–16.
- 1420 [34] M. Fowler, Bluegreendeployment, <http://martinfowler.com/bliki/BlueGreenDeployment.html>, retrieved: August 2016.
- [35] S. Ghobadi, What drives knowledge sharing in software development teams: A literature review and classification framework, *Information & Management*.
- 1425 [36] Tool appendix, retrieved: August 2016.
URL <https://goo.gl/qR6KU5>
- [37] J. M. C. de Gea, J. Nicolas, J. L. F. Aleman, A. Toval, C. Ebert, A. Vizcaino, Requirements engineering tools, *IEEE Software* 28 (4) (2011) 86–91.
- 1430 [38] E. Bjarnason, K. Wnuk, B. Regnell, A case study on benefits and side-effects of agile practices in large-scale requirements engineering, in: *Proceedings of the 1st Workshop on Agile Requirements Engineering, AREW '11*, ACM, New York, NY, USA, 2011, pp. 3:1–3:5.

- [39] M. Dubakov, P. Stevens, Agile tools. the good, the bad and the ugly., Tech. rep., TargetProcess, retrieved: August 2016 (2008).
 1435 URL <http://www.targetprocess.com/download/whitepaper/agiletools.pdf>
- [40] V. Driessen, A successful git branching model, retrieved: November 2015.
 URL <http://nvie.com/posts/a-successful-git-branching-model/>
- [41] A. Pelletier, Another git branching model, retrieved: August 2016.
 1440 URL <http://blogpro.toutantic.net/2012/01/02/another-git-branching-model/>
- [42] R. Fay, A rebase workflow for git, retrieved: August 2016.
 URL <http://randyfay.com/content/rebase-workflow-git>
- [43] D. Spinellis, Version control systems, Software, IEEE 22 (5) (2005) 108–
 1445 109.
- [44] H. Siy, L. Votta, Does the modern code inspection have value?, in: Software Maintenance, 2001. Proceedings. IEEE International Conference on, 2001, pp. 281–289.
- [45] R. K. Yin, Case study research: Design and methods, Sage publications,
 1450 2014.
- [46] G. Guest, K. M. MacQueen, E. E. Namey, Applied thematic analysis, Sage, 2011.
- [47] Tekes — the Finnish funding agency for innovation, retrieved: August 2016.
 URL <http://www.tekes.fi/en/tekes/>

1455 **Appendix A. Interview Outline**

Table A.1: Interview topics. Questions used in this study are shown under the topics.

ID	Related Research Questions	Type	Question
1			Background Information
1.1	–	Closed-ended	How large is your organization?
1.2	–	Open-ended	What is the domain of the project?
1.3	–	Closed-ended	What is the size of the team you represent?
2			Methods
2.1	–	Open-ended	What software engineering methods are you using?
2.2	RQ1, RQ2	Open-ended	Describe and draw your process from requirements to release?
2.3	RQ3	Open-ended	How often do you release?
3			Testing
3.1	RQ1, RQ2	Open-ended	How is the product tested during the development process?
3.2	RQ1, RQ2	Open-ended	What testing tools and frameworks are you using?
4			Operations
5			Toolchain
5.1	RQ1	Open-ended	Add names of the tools used in the process picture drawn at question 2.2.
5.2	RQ2	Open-ended	Are there manual steps in the delivery pipeline? How long do they take?
5.3	RQ1	Open-ended	Do you use life cycle management systems?
6			Life Cycle Management
7			Feedback
7.1	RQ3	Closed-ended	What is your lead time from a customer bug report to release?
7.2	RQ3	Closed-ended	What is your lead time from the event when testing is completed to release?
7.3	RQ3	Closed-ended	How fast can you release a change of one source code line?
8			Delivery Challenges
9			Delivery Benefits